Step 1 — each DNA base rep using 2 bits - for compactness.

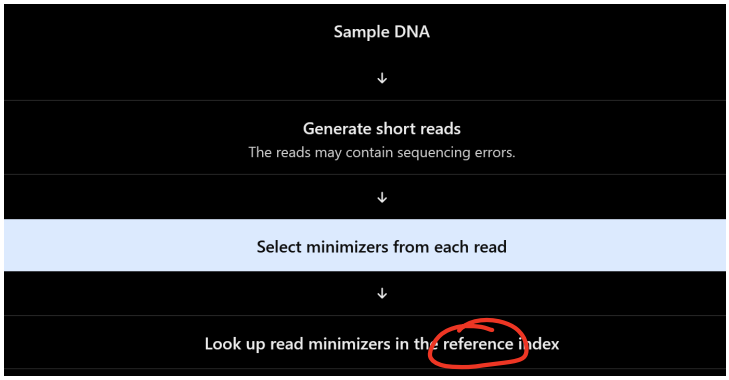
Step 2

Imagine the reference DNA contains thousands of bases. Searching and storing every possible short DNA sequence can be expensive.

Instead, the project selects a smaller set of representative sequences called **minimizers**. These help the program find matching regions efficiently when it processes sequencing reads later.

- k-mer = "k" letter split
a) split into k-mers.
b) each k-mer = HASH VALUE
c) Window of consecutive k-mers.
↓
select the one w/ MIN. # value.

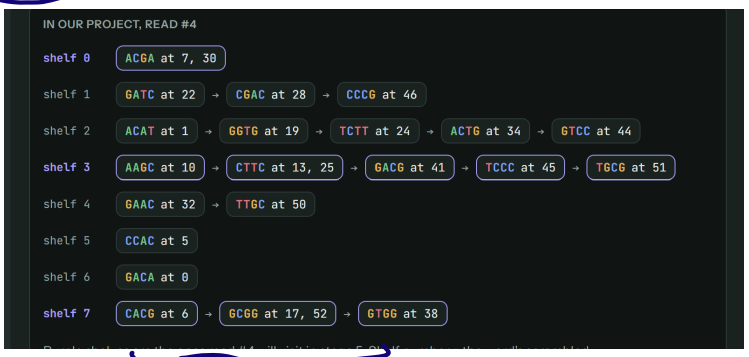
(MP) = selected set of minimizers.



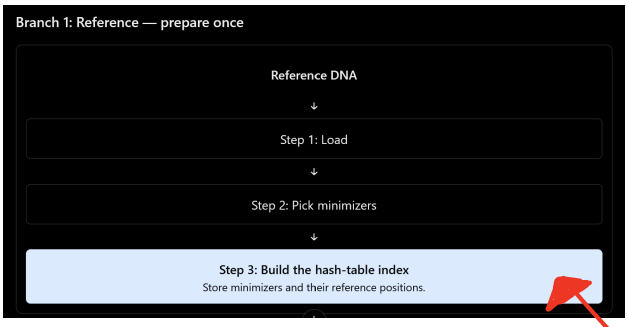
STEP 3 BUILD INDEX (Ref. branch)

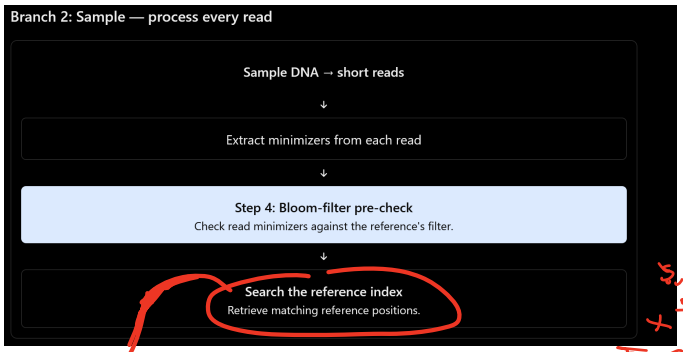
- Store selected minimizers in an "index"
↓
hash table → (old hash) % 8 = giving a number
↓
same num. stored in a BUCKET

Only for reference not sample

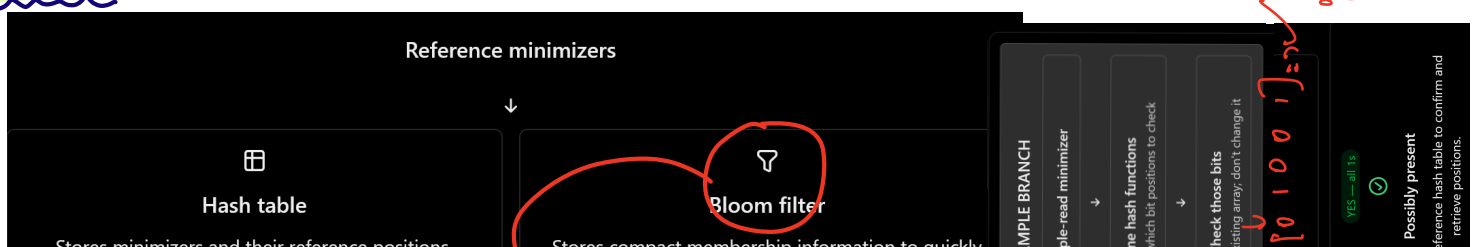


diff minimizer arr.



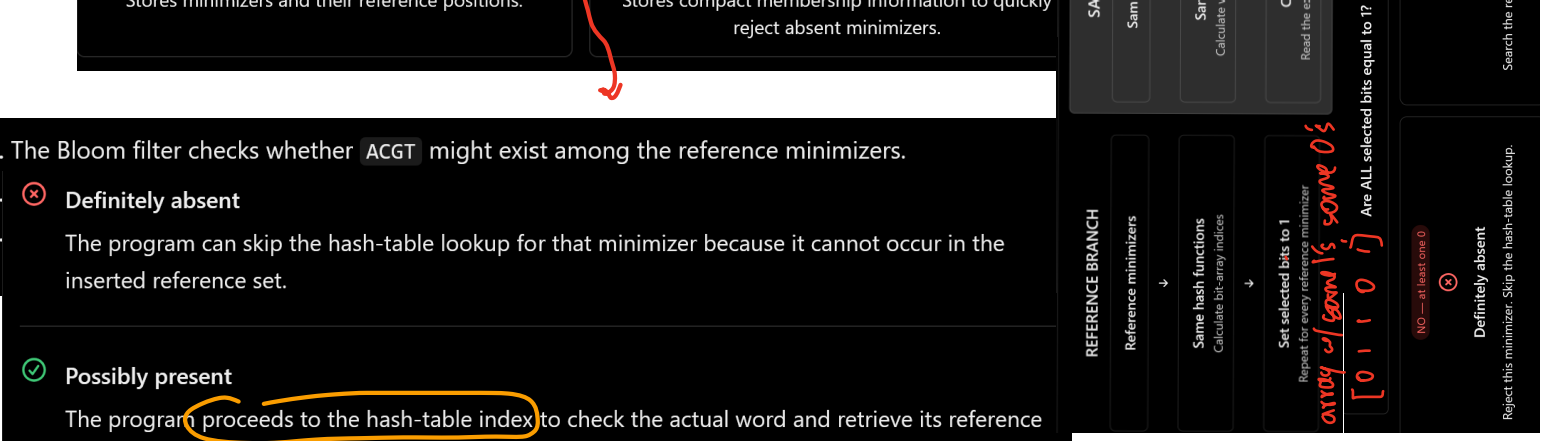


STEP 4 BLOOM FILTER *Ref. branch*



Stores minimizers and their reference positions.

Stores compact membership information to quickly reject absent minimizers.



working of Bloom Filters

STEP 5 VOTE 1.

- sample minimiser goes thru HT. ⇒ we get its LOCN in REFERENCE
- Sample read has a bunch of minimizers.
  - for each, we have reference locn

READ start locn = (locn of MINIMIZER in Reference) − (posn of minimizer in read)

obtained for each minimizer in the read.

∴ multiple possible st locns obtained

⇓

VOTING (counts obtained)

Minimizer	Bloom-filter result	What happens?
A	Possibly present	Query the reference hash table
B	Definitely absent	Skip this minimizer's index lookup
C	Possibly present	Query the reference hash table
D	Possibly present	Query the reference hash table
E	Definitely absent	Skip this minimizer's index lookup

Only the individual minimizer is skipped, not the entire read.

STEP 6

- Reads st posn obtained.
- Vote counts for all st posns obtained

*TOP 3 candidates chosen in a MIN-HEAP.

- binary tree
- holds max. 3 elements.
- Lowest vote on top
- if new candidate > Lowest current,

↓

it replaces it.

STEP 7 VERIFY MATCHING.

- Check letter for letter whether sample read actually matches the.

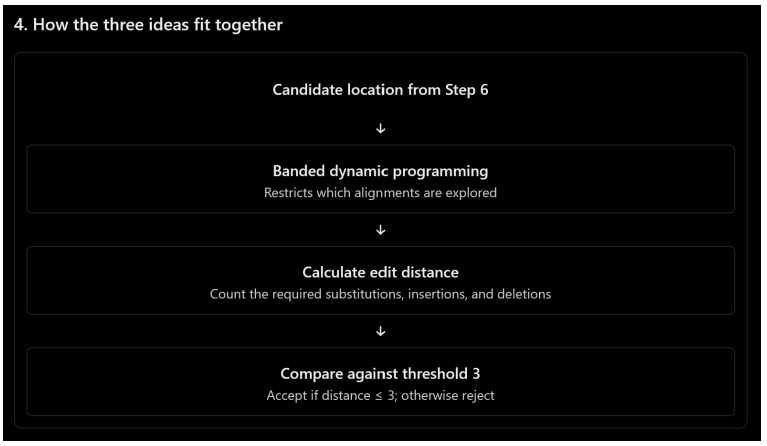
reference read.

· Might still not match even after all this filtering

BANDED DYNAMIC PGMING

DP → A 2d matrix w/ each cell showing corresponding edit distance.
→ But checks every cell to find min. edit dist

BANDED DP | Only a BAND of cells around diagonal of matrix checked (less computation)

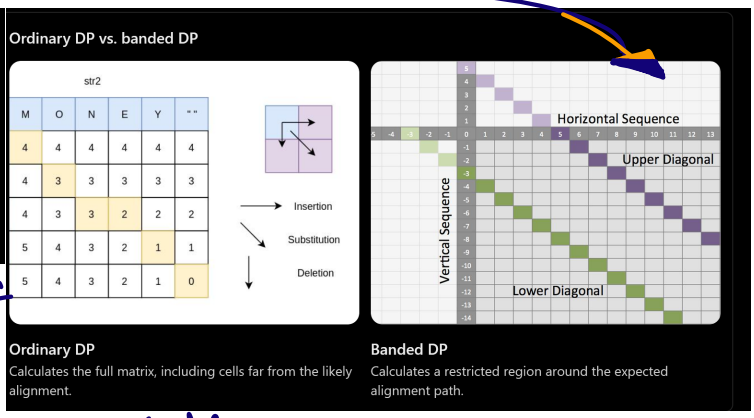


(

The complete DP matrix

Read \ Reference	–	A	G	C	T
–	0	1	2	3	4
A	1	0	1	2	3
G	2	1	0	1	2
T	3	2	1	1	1

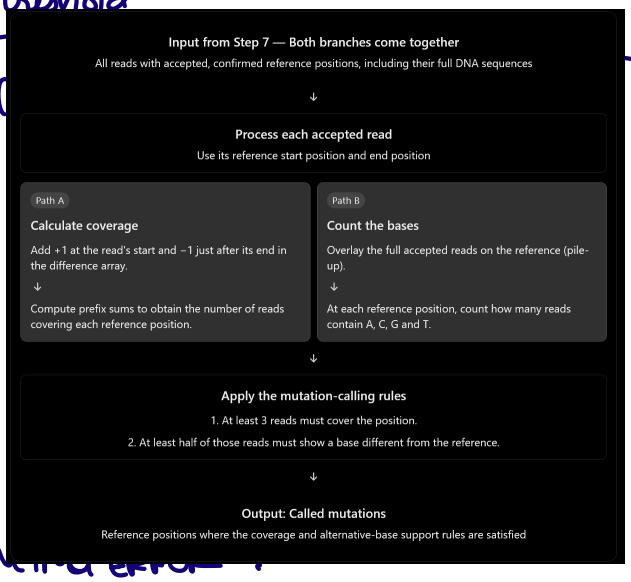
The final answer is the bottom-right cell: 1.



* decides whether a read actually present at pos" told by step 6.
* Filter out everything that has edit dist -> threshold

STEP 8 } - inp: step 7: confirmed posns.

MUTATION FINDING



* Compares all reads to reference for a ptc location.

• If huge no. of reads all report diff. letter comp to reference => MUTATION

• If only few reads show diff. value => SEQUENCING ERROR

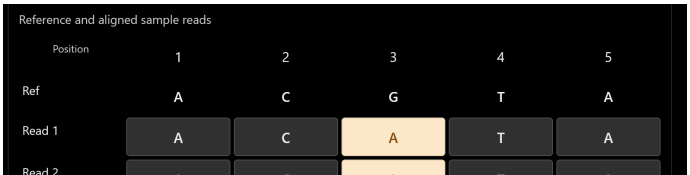
+RULE → At least (3) aligned reads cover that portion → >half the reads support that anomaly } then MUTATION else ERROR

. Coverage => No. of reads that "cover" a portion of reference

For a read covering positions 2 through 4:

- Add `+1` at position 2.
- Add `-1` immediately after its end, at position 5.
- Take a prefix sum across the array.

Example: one read covers positions 2–4



Position	1	2	3	4	5	6
Difference	0	1	0	0	-1	0
Coverage	0	1	1	1	0	0

The prefix sum converts the two boundary updates into coverage at every position.

This works because the `+1` starts the coverage contribution and the `−1` cancels it just after the read ends.

Read 3	A	C	A	T	A

At reference position 3, the reference has G, while all three illustrative reads have A.

Coverage

STEP 9 { EVALUATION

- Check step 8 prediction against actual data.
- Only possible for obs. existing info and mutation.